

Вариативное задание 2.1. Исследовать возможности сервиса GitLab для управления временем.

1. Цель и задачи работы

Цель работы – исследовать возможности сервиса GitLab для управления временем (тайм-менеджмента) в контексте задач разработчика и оценить удобство и эффективность работы с ним.

Для достижения цели были поставлены следующие задачи:

- выбрать сервис и обосновать выбор;
- перенести в сервис реальный учебный проект;
- изучить и применить основные функции планирования: метки, вехи, задачи, учёт времени, доску задач, связь задач с кодом;
- спланировать дальнейшую работу над проектом;
- выделить достоинства и недостатки сервиса и оценить его удобство и эффективность.

2. Выбор сервиса

Из двух предложенных сервисов, GitHub и GitLab, для исследования был выбран GitLab. Основная причина – встроенный учёт времени, доступный на бесплатном тарифе: для каждой задачи можно указать плановую оценку и фактически затраченное время, а сервис сам сравнивает эти значения и суммирует их по этапам проекта. В GitHub такой функции нет, время приходится учитывать через сторонние интеграции. Для задания, посвящённого именно управлению временем, это решающий аргумент.

Работа выполнялась на платформе `gitlab.com` (бесплатный тариф). Университетский сервер `git.herzen.spb.ru` построен на той же системе, поэтому описанные функции доступны и там, с поправкой на версию.

3. Объект исследования

Чтобы исследование опиралось на реальные задачи, а не на искусственный пример, в GitLab был перенесён собственный проект –

мобильное приложение для автоматизации процессов в кафе. Приложение разрабатывается на Flutter (Dart) для Android и iOS; серверную часть планируется написать на Rust и развернуть в Docker на домашнем Ubuntu-сервере.

Проект начат в августе 2026 года. К концу августа была готова локальная версия модуля складского учёта, работающая только на телефоне, и с её помощью проведён тестовый пересчёт склада в кафе. Впереди – разработка серверной части, синхронизация данных между устройствами и доработка интерфейса.

Основной репозиторий проекта находится на GitHub. Он был импортирован в GitLab встроенным инструментом импорта с авторизацией через токен GitHub; вместе с кодом переносится история коммитов, ветки, а также задачи, pull request'ы, метки и вехи, если они есть в исходном репозитории. Задачи прошедших этапов (август 2026 года) внесены в GitLab ретроспективно, фактически затраченное время указано по памяти.

4. Основные функции GitLab и их применение

4.1. Метки (Labels)

Метки – цветные ярлыки, которые присваиваются задачам и merge request'ам. Они позволяют классифицировать задачи и фильтровать их в списках и на доске. Метки создаются в разделе Manage → Labels (рисунки 1 и 2).

Для проекта были созданы метки по нескольким признакам: часть системы (app, backend, infra), модуль приложения (склад, техкарты), тип работы (изучение), приоритет (high) и статус (Doing – задача в работе, используется на доске).

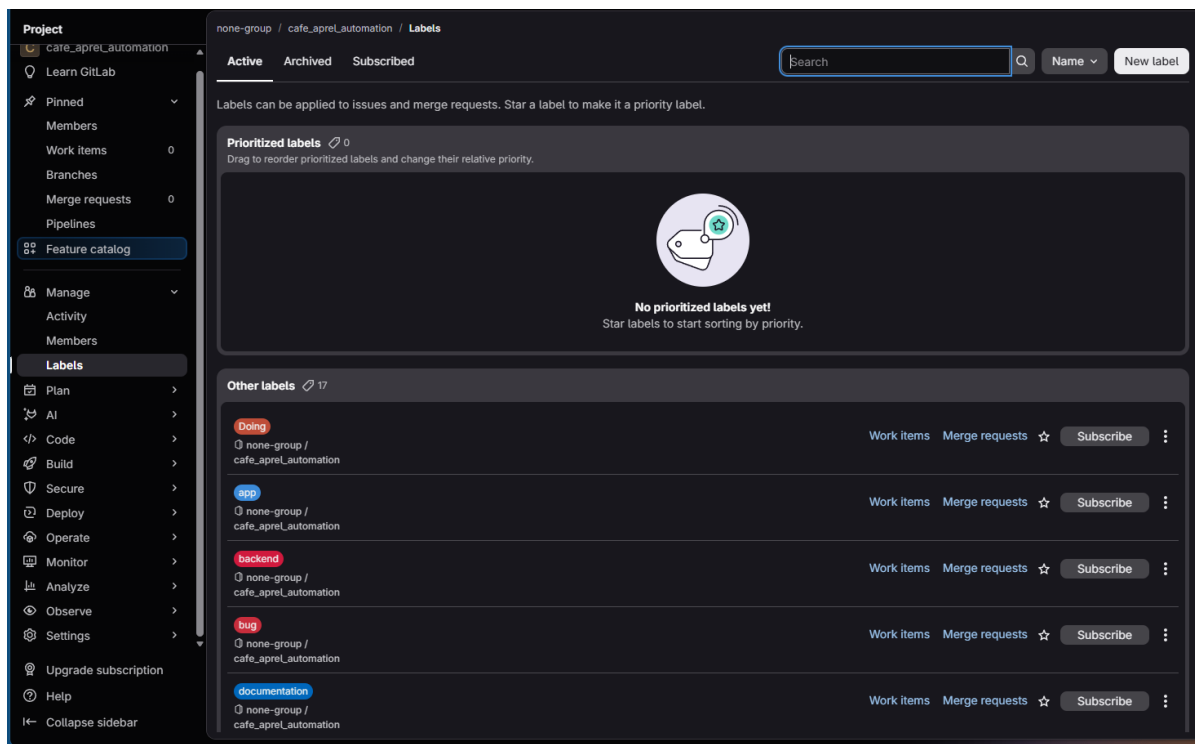


Рисунок 1 – Список меток проекта

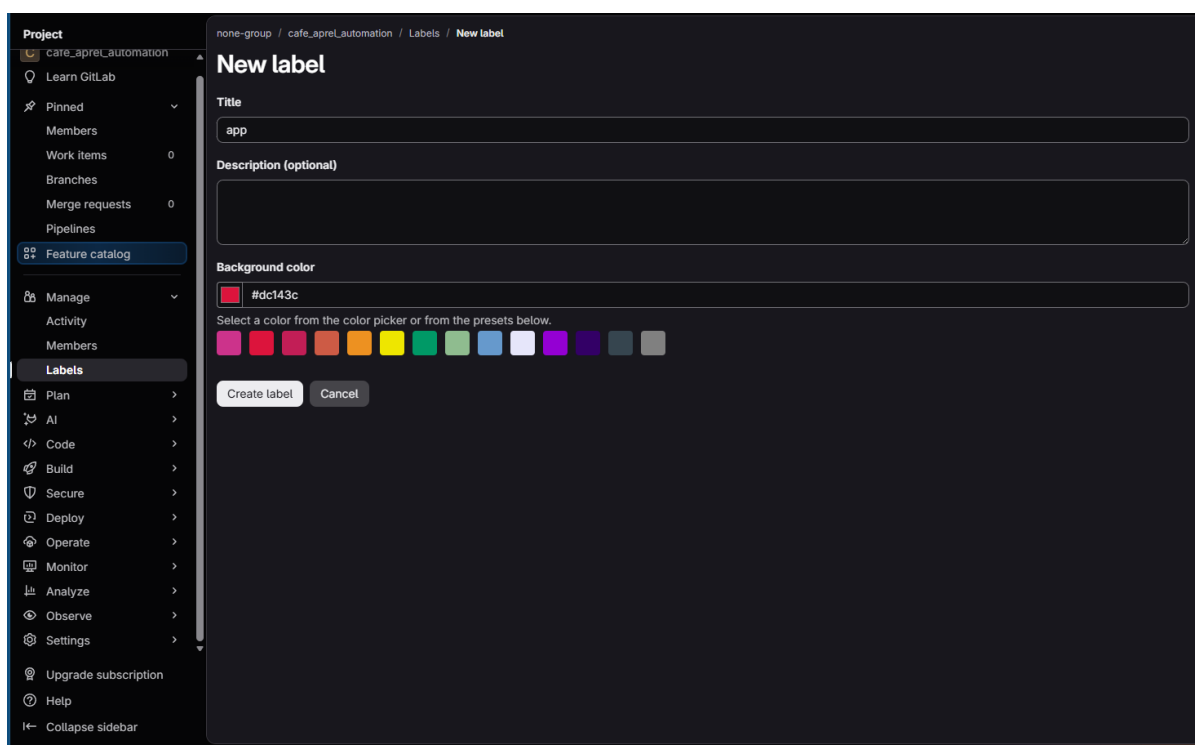


Рисунок 2 – Создание новой метки

4.2. Вехи (Milestones)

Веха – это этап проекта с датами начала и окончания, аналог спринта. Задачи привязываются к вехе, после чего GitLab показывает процент её выполнения, список задач по статусам и суммарное плановое и фактическое время. Вехи создаются в разделе Plan → Milestones (рисунки 3-6).

Работа над проектом была разбита на четыре этапа:

- **M0 Инфраструктура** (01.08-22.08.2026) – изучение Flutter, создание проекта и репозитория;
- **M1 Склад: локальная версия** (22.08-31.08.2026) – экраны склада, локальное хранение данных, тестовый пересчёт;
- **M2 Склад: бэкенд и синхронизация** (01.10-20.12.2026) – серверная часть, синхронизация, доработка интерфейса;
- **M3 Техкарты** (без сроков) – следующий модуль, запланирован на будущее.

Завершённые этапы M0 и M1 закрыты, этап M2 является текущим планом работы (рисунок 7).

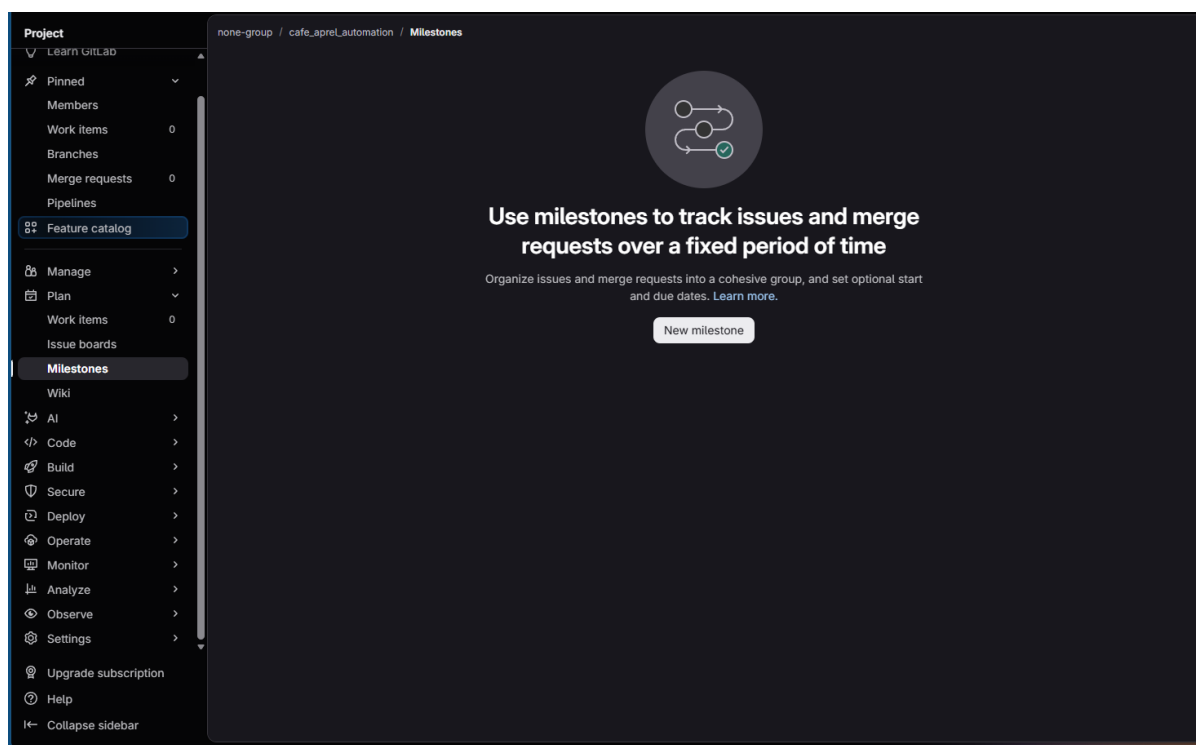


Рисунок 3 – Раздел вех до создания первой вехи

Project: none-group / cafe_apreAutomation / Milestones / New milestone

New milestone

Title

M0 Инфраструктура

Start Date

2026-08-01

[Clear start date](#)

Due Date

2026-08-22

[Clear due date](#)

Description

Normal text | B | I | S | L | U | P | M | A | H | E | O | G | +

Подготовка к разработке: структура репозитория, изучение Flutter и Rust, создание проекта

[Switch to plain text editing](#)

[Create milestone](#) [Cancel](#)

Рисунок 4 – Создание вехи M0 с датами начала и окончания

Project: none-group / cafe_apreAutomation / Milestones / New milestone

New milestone

Title

M2 Склад: бэкенд и синхронизация

Start Date

2026-10-01

[Clear start date](#)

Due Date

2026-12-20

[Clear due date](#)

Description

Normal text | B | I | S | L | U | P | M | A | H | E | O | G | +

Задачи:

- изучить основы Rust
- модель данных склада и БД на сервере
- API склада на Rust
- запустить бэкенд в Docker на домашнем сервере
- синхронизация по ревизиям (приоритет high)
- доработка интерфейса склада

[Switch to plain text editing](#)

[Create milestone](#) [Cancel](#)

Рисунок 5 – Создание вехи M2 – текущего этапа работы

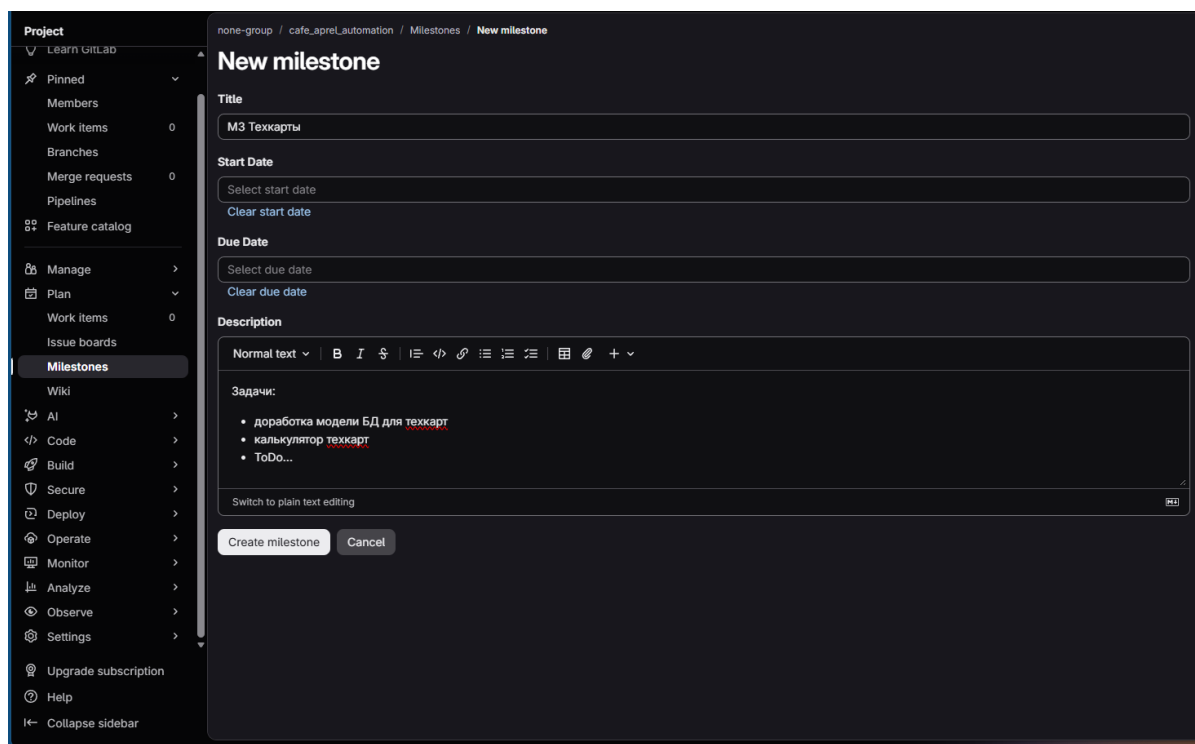


Рисунок 6 – Создание вехи М3 без сроков

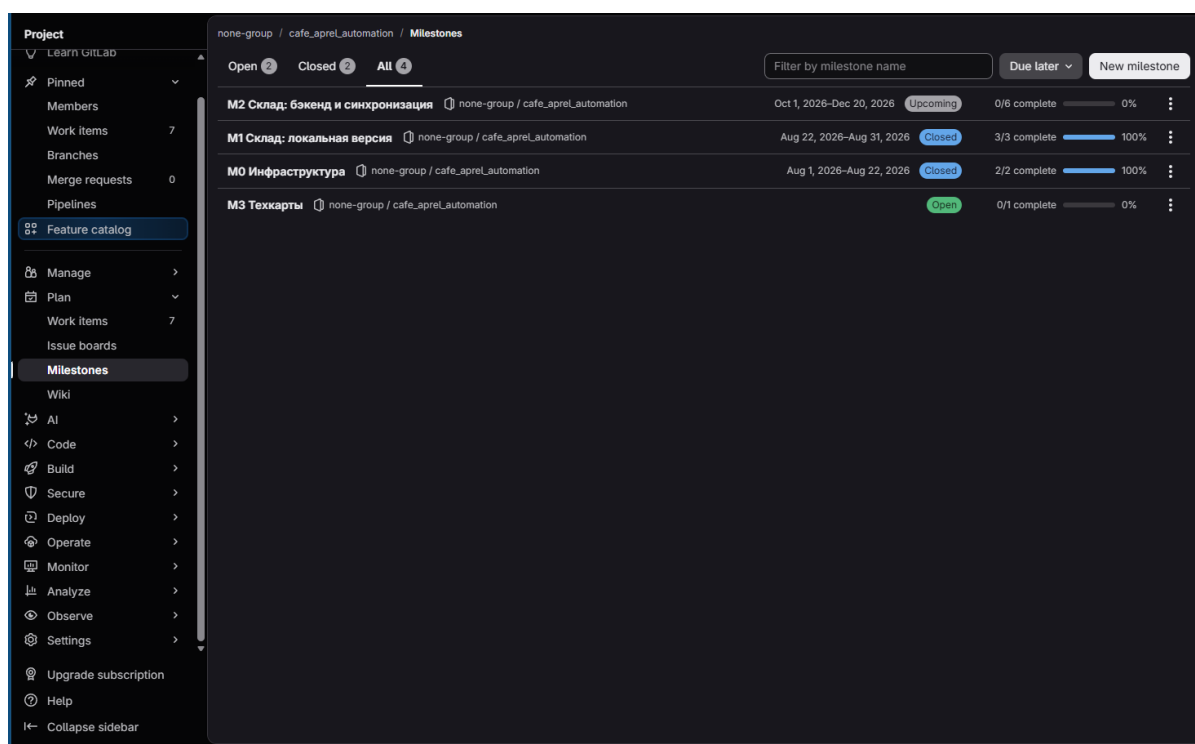


Рисунок 7 – Список вех проекта с процентом выполнения

4.3. Задачи (Work items / Issues)

Задача – основная единица планирования. В актуальной версии GitLab задачи находятся в разделе Plan → Work items. При создании задачи указываются заголовок, описание, исполнитель, метки, веха и сроки (рисунки 8 и 9). В описании можно вести чек-лист подзадач.

Кроме полей формы, GitLab поддерживает быстрые команды (quick actions) – строки вида /label, /milestone, /estimate, /spend, которые вписываются в описание или комментарий и выполняются при сохранении. Это ускоряет работу, но команды должны стоять каждая на отдельной строке; в визуальном режиме редактора переносы строк при вставке текста терялись, и команды не срабатывали. Поэтому в работе параметры задач задавались в основном через боковую панель.

Всего в проекте создано 13 задач: 5 по завершённым этапам M0 и M1, 7 по текущему этапу M2 и 1 по этапу M3.

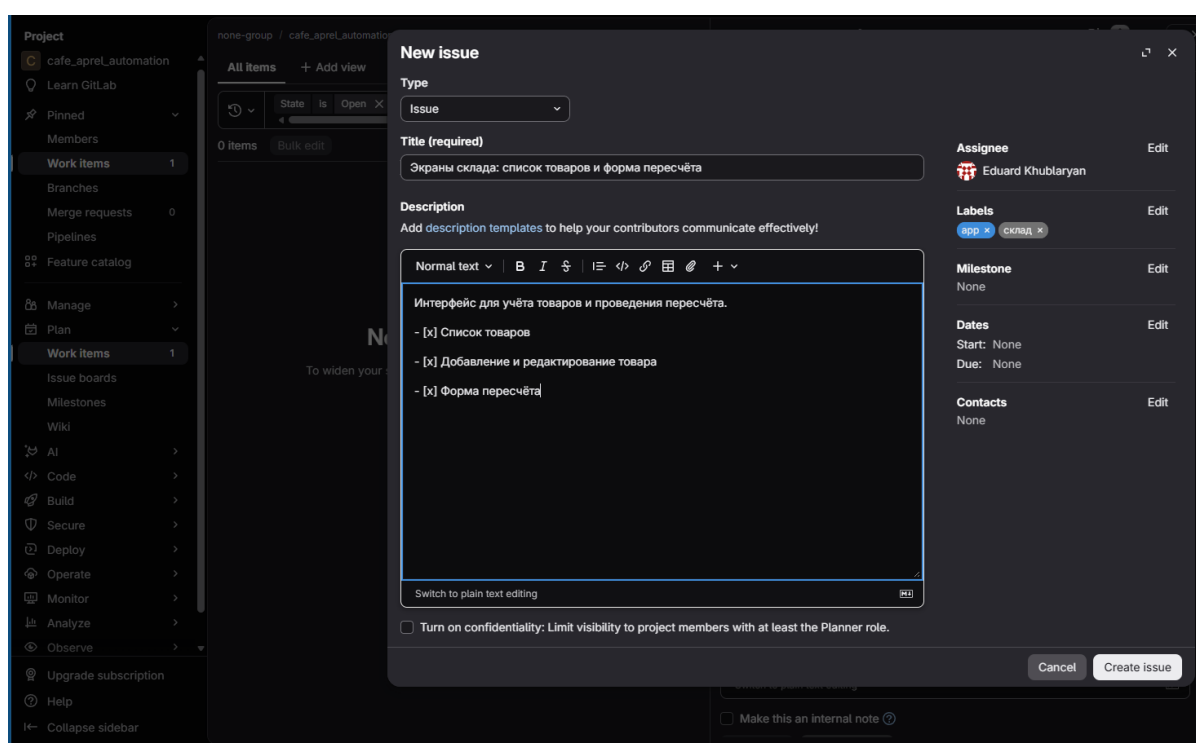


Рисунок 8 – Форма создания задачи

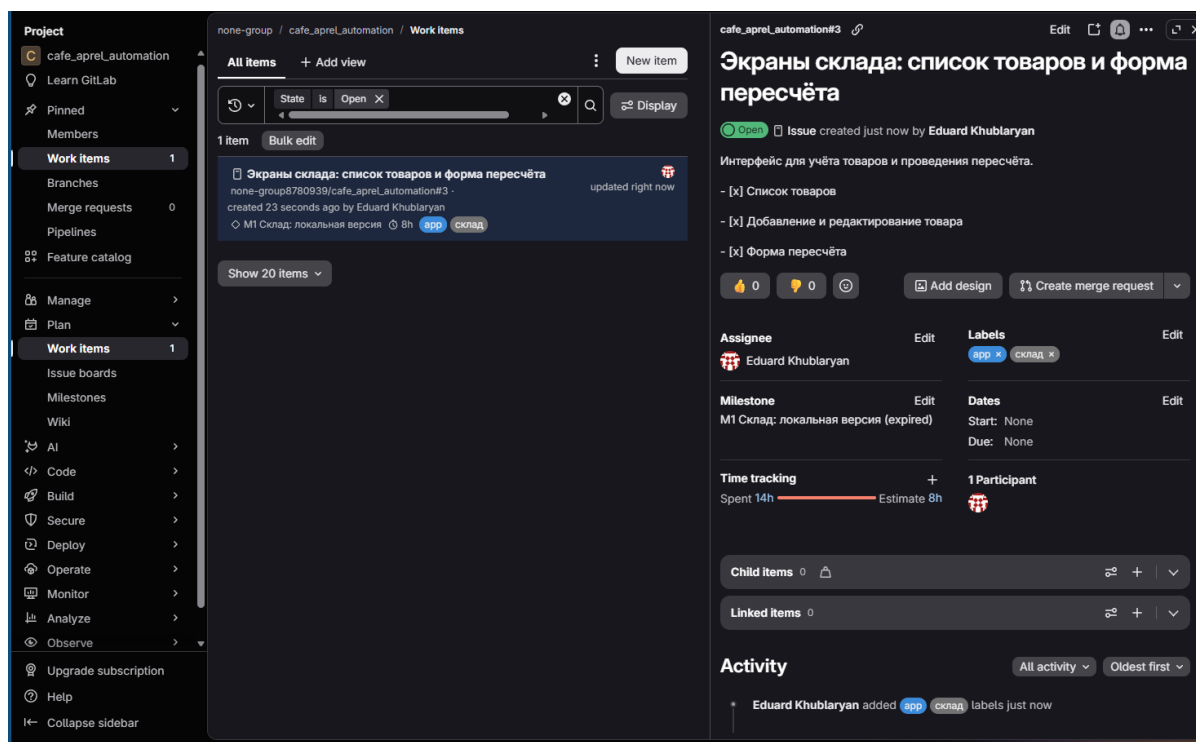


Рисунок 9 – Карточка задачи: метки, веха, учёт времени

4.4. Учёт времени (Time tracking)

Для каждой задачи задаётся плановая оценка (Estimate) и записывается фактически затраченное время (Spent). Время добавляется кнопкой «+» в блоке Time tracking или командой /spend, каждая запись сохраняется в истории задачи (рисунок 10). В карточке задачи отображается полоса «затрачено / запланировано», а на странице вехи – суммарные значения по всем её задачам (рисунок 11).

Именно эта функция дала самый полезный результат работы: сравнение плана с фактом по завершённомu этапу (см. раздел 5).

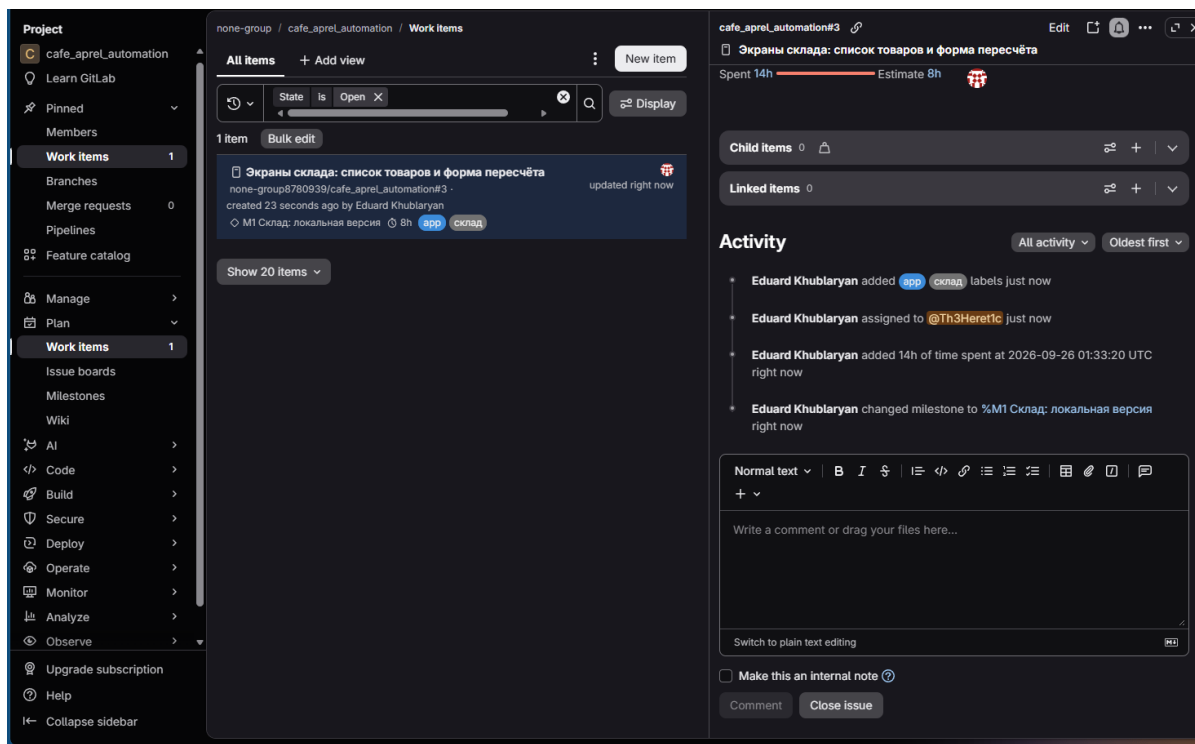


Рисунок 10 – История задачи: запись затраченного времени и изменение параметров

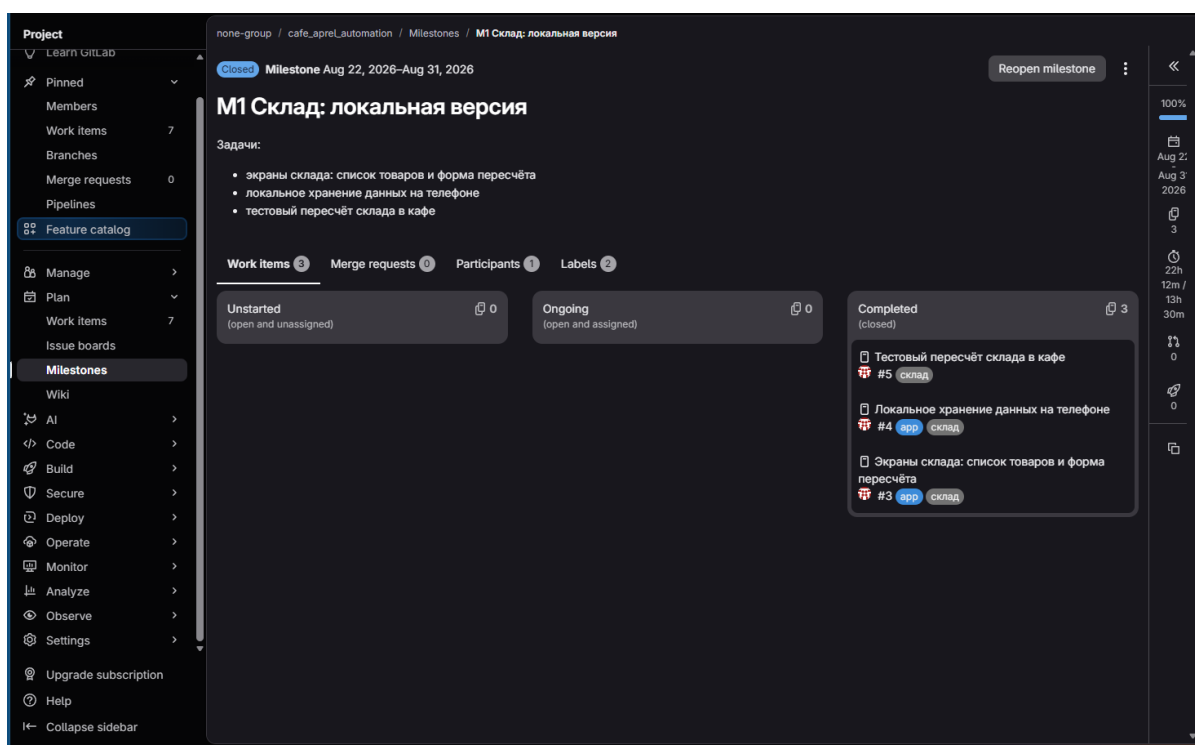


Рисунок 11 – Страница завершённой вехи M1: задачи по статусам, выполнение 100%, время

4.5. Доска задач (Issue boards)

Доска (Plan → Issue boards) показывает задачи в виде канбан-карточек, распределённых по колонкам. По умолчанию есть колонки Open и Closed; дополнительно была создана колонка по метке Doing для задач, находящихся

в работе. При перетаскивании карточки между колонками метка меняется автоматически. На карточках видны метки, веха, срок и оценка времени (рисунок 12).

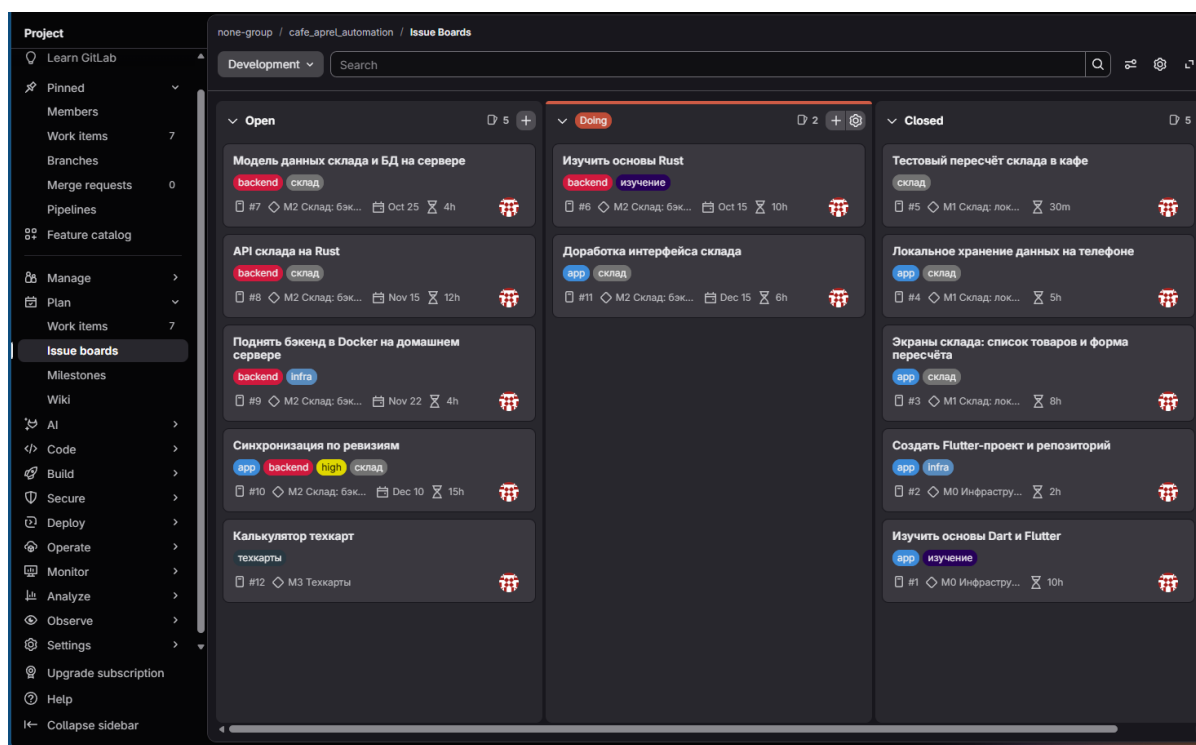


Рисунок 12 – Доска задач проекта

4.6. Связь задач с кодом (Merge requests)

Отличие GitLab от обычных планировщиков – связь задач с кодом. Merge request (аналог pull request в GitHub) – это запрос на слияние изменений из отдельной ветки в основную. Если создать его со страницы задачи, GitLab автоматически создаёт ветку и связывает её с задачей, а после слияния закрывает задачу.

Функция была проверена на реальной задаче: шаблонный README проекта был заменён описанием приложения, его стека, текущего статуса и плана развития. Была создана задача «Обновить README: описание проекта и план развития», из неё – merge request, изменения внесены в созданную ветку и слиты в основную. Задача закрылась автоматически, а на её странице появилась ссылка на merge request (рисунки 13 и 14).

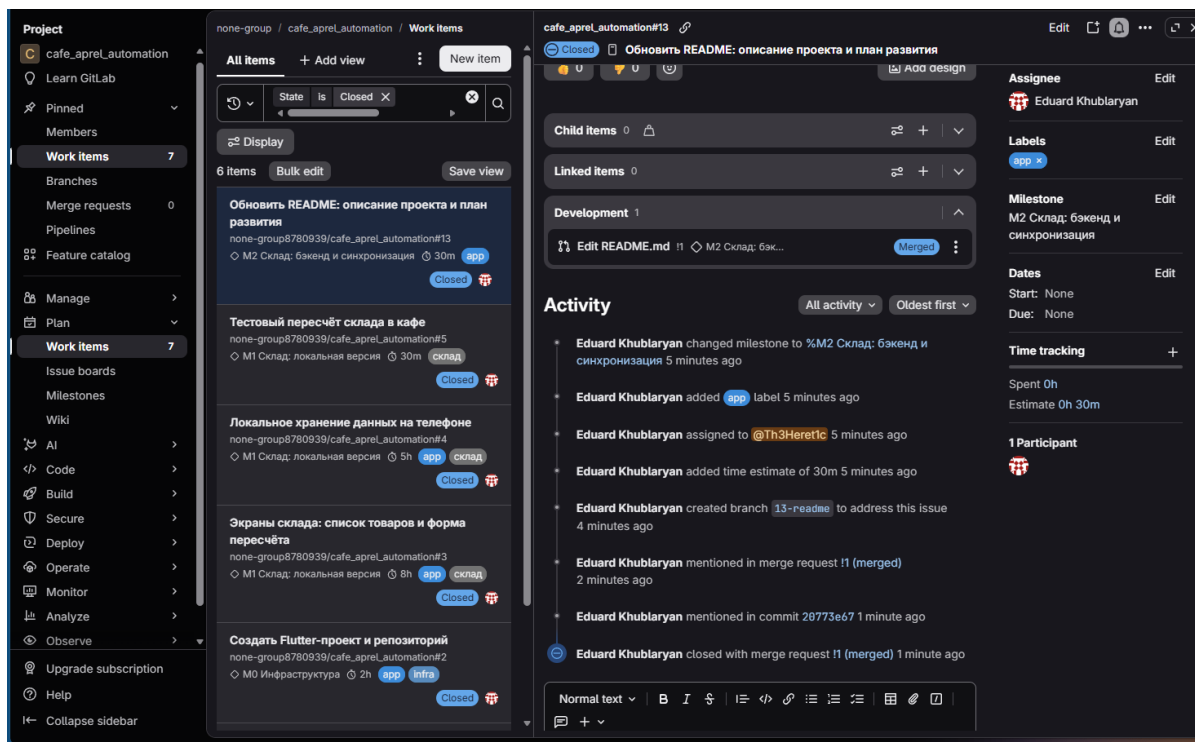


Рисунок 13 – Задача, автоматически закрываемая через merge request

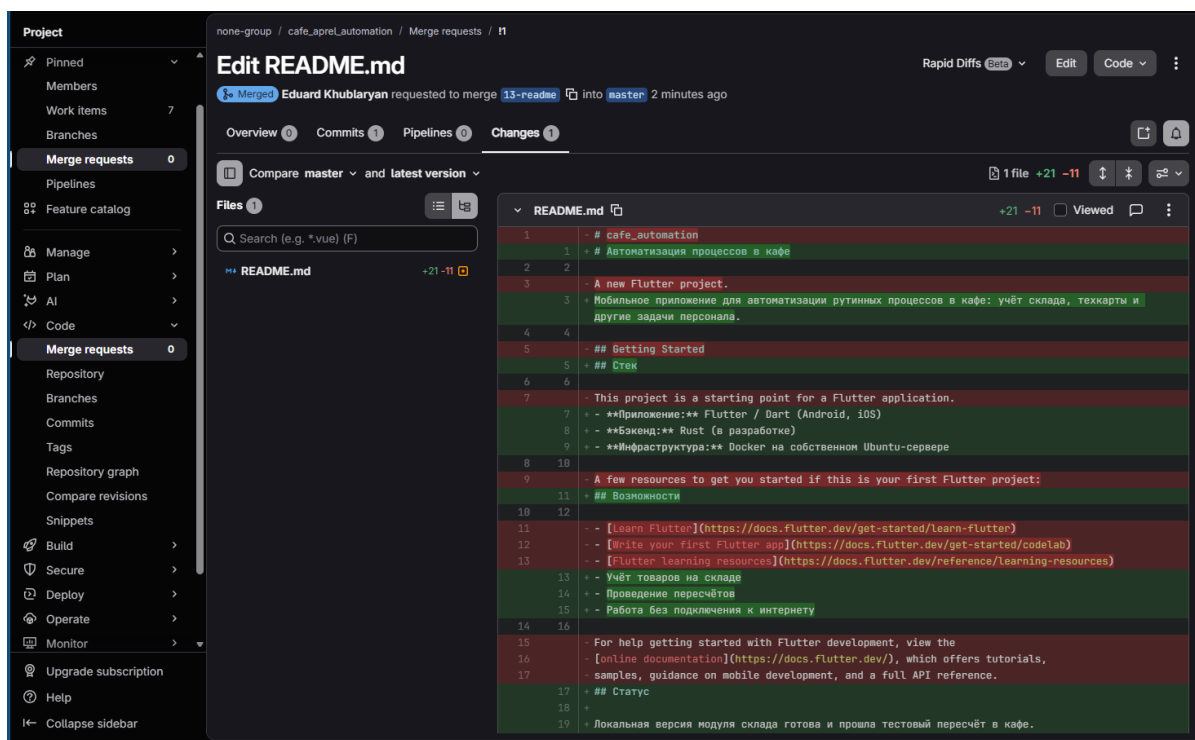


Рисунок 14 – Изменения в merge request

5. Результаты использования

По итогам работы в GitLab сформирован план проекта, в котором видны и выполненная работа, и предстоящая. Сводные данные по этапам приведены в таблице 1.

Таблица 1 – Состояние этапов проекта

Этап	Сроки	Задачи	Статус
M0 Инфраструктура	01.08–22.08.2026	2 из 2	Закрыт, 100%
M1 Склад: локальная версия	22.08–31.08.2026	3 из 3	Закрыт, 100%
M2 Склад: бэкенд и синхронизация	01.10–20.12.2026	1 из 7	В работе
M3 Техкарты	не заданы	0 из 1	Запланирован

По этапу M1 плановая оценка составила 13 ч 30 мин, фактически затрачено около 22 ч – примерно в 1,6 раза больше плана. Недооценка объясняется тем, что Flutter был новой технологией, и значительная часть времени ушла на изучение и исправление ошибок.

План текущего этапа M2 приведён в таблице 2. Суммарная оценка открытых задач – 51 ч. С учётом опыта этапа M1 реалистичный объём работ ближе к 80 ч, и это стоит учитывать при планировании: этап M2 также связан с освоением новой технологии (Rust).

Таблица 2 – План этапа M2

Задача	Срок	Оценка	Статус
Изучить основы Rust	15.10.2026	10 ч	В работе
Модель данных склада и БД на сервере	25.10.2026	4 ч	Открыта
API склада на Rust	15.11.2026	12 ч	Открыта
Поднять бэкенд в Docker на домашнем сервере	22.11.2026	4 ч	Открыта
Синхронизация по ревизиям (приоритет high)	10.12.2026	15 ч	Открыта
Доработка интерфейса склада	15.12.2026	6 ч	В работе
Обновить README	–	30 мин	Выполнена

6. Достоинства и недостатки

6.1. Достоинства

- **Встроенный учёт времени на бесплатном тарифе.** Оценка и факт по каждой задаче и суммарно по этапу, без сторонних сервисов.
- **Связь задач с кодом.** Ветки и merge request'ы создаются из задачи, задача закрывается автоматически после слияния – задачи и работа над кодом находятся в одном месте.

- **Наглядность.** Вехи показывают процент выполнения, доска – текущее состояние работы, чек-листы – прогресс внутри задачи.
- **Простой перенос проекта.** Импорт из GitHub переносит не только код, но и задачи, метки и вехи.
- **Гибкая классификация.** Метки позволяют группировать задачи по любым признакам и фильтровать их.
- **Доступность.** Университет использует собственный сервер GitLab, поэтому инструмент доступен в учебном процессе.

6.2. Недостатки

- **Перегруженный интерфейс.** Много разделов, часть из которых не нужна для планирования; новичку сложно сразу найти нужную функцию. На экране постоянно присутствует рекламная панель ИИ-ассистента GitLab Duo.
- **Особенности редактора.** В визуальном режиме при вставке текста терялись переносы строк: не срабатывали быстрые команды, а чек-листы отображались как обычный текст. Приходилось переключаться в режим plain text.
- **Ограничения бесплатного тарифа.** Итерации, диаграммы сгорания задач (burndown charts), взаимоисключающие (scoped) метки доступны только в платных тарифах.
- **Неочевидные значения по умолчанию.** При записи затраченного времени дата по умолчанию – текущая; её легко не поменять. Дату создания задачи задним числом установить нельзя.
- **Нет напоминаний на телефоне.** У GitLab нет официального мобильного приложения, поэтому напоминать о сроках так, как это делают планировщики вроде Microsoft To Do, он не может.

7. Оценка удобства и эффективности

Удобство. На освоение базовых функций ушло несколько часов, в основном из-за объёма интерфейса и особенностей редактора. После этого создание и ведение задач через боковую панель стало простым. Для личных

бытовых дел GitLab избыточен, но для учебных и собственных проектов разработчика он удобнее обычного планировщика, так как задачи хранятся рядом с кодом.

Эффективность. Главный практический результат – возможность сравнить план с фактом. Данные по этапу M1 показали систематическую недооценку задач примерно в 1,6 раза; без учёта времени этот вывод не был бы очевиден. Он используется при планировании этапа M2. Кроме того, разбиение проекта на вехи с конкретными задачами и сроками превратило общее намерение «доделать бэкенд» в последовательный план с датами.

Итоговая оценка: GitLab эффективен как инструмент планирования и контроля работы над программным проектом, особенно при регулярной записи затраченного времени. Для полноценного тайм-менеджмента в течение дня его стоит дополнять календарём или приложением-напоминалкой.

Заключение

В ходе работы были изучены и применены основные функции GitLab для управления временем разработчика: метки, вехи, задачи с чек-листами, учёт времени, доска задач и связь задач с кодом через merge request'ы. На примере реального проекта мобильного приложения для кафе сформирован план работ с этапами и сроками, по завершённомому этапу получены данные о расхождении оценки и фактических затрат, а по текущему этапу составлен план на октябрь–декабрь 2026 года. Сервис признан полезным и будет использоваться при дальнейшей работе над проектом.